```
    canvasPath(sampleData[x]);
    context.stroke()
    context.fill();
  }
};

function zoomInitialized() {
  d3.selectAll("path.sample")
    .style("display", "none");
  zoomed();
};

function zoomFinished() {
  var context = d3.select("canvas").node().getContext("2d");
  context.clearRect(0,0,500,500);
  canvasPath.context(context)

  context.strokeStyle = "black";
  context.fillStyle = "gray";
  context.lineWidth = "1px";
  for (var x in countries.features) {
    context.beginPath();
    canvasPath(countries.features[x]);
    context.stroke()
    context.fill();
  }

  d3.selectAll("path.sample")
    .style("display", "block")
    .attr("d", svgPath);
};
```

Hides SVG elements when zooming starts

Calls zoomed to draw with canvas the SVG triangles we just hid

Only draws countries with canvas at the end of the zoom

Shows SVG elements when zoom ends

Sets the new position of SVG elements

As a result of this new code, we have a map that uses canvas rendering when users zoom and pan, but SVG rendering when the map is fixed in place and users have the ability to click, mouse over, or otherwise interact with the graphical elements. It's the best of both worlds. The only drawback of this approach is that we have to invest more time making sure our `<canvas>` element and our `<svg>` element line up perfectly, and that our opacity, fill colors, and so on are close enough matches that it's not jarring to the user to see the different modes. I haven't done this in the previous code, so that you can see that the two modes are in operation at the same time, and that's reflected in the difference between the two graphical outputs in figure 11.6.

The kind of pixel-perfect alignment necessary to make the transition from one mode to another, as well as the fastidious color matching also required, isn't something I have the space to explain in this book, but you'll need to do both to make the best interactive information visualization. If you look closely at figure 11.6, you'll notice that the canvas element (on the right) is a pixel or so shifted up and to the left, and that's without testing it in other browsers that may have different default settings for `<canvas>` or `<svg>` or both.

Finally, using canvas and SVG drawing simultaneously may present a difficulty. Say we want to draw a canvas layer over an SVG layer because we want the canvas layer to